Lab 7: Fuzzing for Input Validation Bugs

Name: Nikita Mankani **SRN:** PES1UG24CS300 **Repo:** 54_museum_admission_L6_L7 (Problem #54 – Museum Admission)

Module under test: `museum_admission_parser.py` **Tests:** `test_museum_admission_parser.py` (Hypothesis + pytest)

SS1 – Buggy module: `python museum_admission_parser.py`

Valid input works, but a blank name, a non-numeric age ("twelve") and a date in the wrong format ("05/11/2026") each crash the program with an unhandled exception.

```
LAB7 — zsh

$ python museum_admission_parser.py
==== Running format_visitor_name ====
Enter a visitor's full name (e.g. asha rao): asha rao
Name on ticket: Rao, Asha

==== Running parse_group_ages ====
Enter the ages in the group separated by spaces (e.g. 34 12 70): 34 12 70
Group ages: [34, 12, 70]

==== Running parse_visit_date ====
Enter the visit date as YYYY-MM-DD (e.g. 2026-11-05): 2026-11-05
Visit date (year, month, day): (2026, 11, 5)

$ python museum_admission_parser.py
==== Running format_visitor_name ====
Enter a visitor's full name (e.g. asha rao):
Traceback (most recent call last):
  File "~/LAB7/museum_admission_parser.py", line 38, in <module>
    main()
    ~~~~^^
  File "~/LAB7/museum_admission_parser.py", line 27, in main
    print("Name on ticket:", format_visitor_name(value))
                             ~~~~~~~~~~~~~~~~~~~^^^^^^^
  File "~/LAB7/museum_admission_parser.py", line 7, in format_visitor_name
    return parts[-1].title() + ', ' + ' '.join(parts[:-1]).title()
           ~~~~~^^^^
IndexError: list index out of range

$ python museum_admission_parser.py
==== Running format_visitor_name ====
Enter a visitor's full name (e.g. asha rao): asha rao
Name on ticket: Rao, Asha

==== Running parse_group_ages ====
Enter the ages in the group separated by spaces (e.g. 34 12 70): 34 twelve 70
Traceback (most recent call last):
  File "~/LAB7/museum_admission_parser.py", line 38, in <module>
    main()
    ~~~~^^
  File "~/LAB7/museum_admission_parser.py", line 31, in main
    print("Group ages:", parse_group_ages(value))
                        ~~~~~~~~~~~~~~~~~^^^^^^^
  File "~/LAB7/museum_admission_parser.py", line 14, in parse_group_ages
    return [int(age) for age in text.split()]
            ~~~^^^^^
ValueError: invalid literal for int() with base 10: 'twelve'

$ python museum_admission_parser.py
==== Running format_visitor_name ====
Enter a visitor's full name (e.g. asha rao): asha rao
Name on ticket: Rao, Asha

==== Running parse_group_ages ====
Enter the ages in the group separated by spaces (e.g. 34 12 70): 34 12 70
Group ages: [34, 12, 70]

==== Running parse_visit_date ====
Enter the visit date as YYYY-MM-DD (e.g. 2026-11-05): 05/11/2026
Traceback (most recent call last):
  File "~/LAB7/museum_admission_parser.py", line 38, in <module>
    main()
    ~~~~^^
  File "~/LAB7/museum_admission_parser.py", line 35, in main
    print("Visit date (year, month, day):", parse_visit_date(value))
                                            ~~~~~~~~~~~~~~~~~^^^^^^^
  File "~/LAB7/museum_admission_parser.py", line 21, in parse_visit_date
    year, month, day = text.strip().split('-')
    ^^^^^^^^^^^^^^^^
ValueError: not enough values to unpack (expected 3, got 1)
```

SS2 – Hypothesis tests failing on the buggy module: `python -m pytest test_museum_admission_parser.py`

All 3 property-based tests fail. Hypothesis shrinks each failure to a minimal input (shown under "Failing test case").



```
    |     ), f"parse_group_ages returned an invalid age list for {s!r}: {result!r}"
    | AssertionError: parse_group_ages returned an invalid age list for '-1': [-1]
    | assert ([-1] is None or (True and False))
    |  +  where True = isinstance([-1], list)
    |  +  and   False = all(<generator object test_parse_group_ages_no_crash.<locals>.<genexpr> at 0x7f3fdaae7bc0>)
    | Failing test case: test_parse_group_ages_no_crash(
    |     s='-1',
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         ~/LAB7/test_museum_admission_parser.py:37
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "~/LAB7/test_museum_admission_parser.py", line 31, in test_parse_group_ages_no_crash
    |     result = parse_group_ages(s)
    |   File "~/LAB7/museum_admission_parser.py", line 14, in parse_group_ages
    |     return [int(age) for age in text.split()]
    |             ~~~^^^^^
    | ValueError: invalid literal for int() with base 10: ':'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "~/LAB7/test_museum_admission_parser.py", line 33, in test_parse_group_ages_no_crash
    |     assert False, f"parse_group_ages crashed on {s!r} with: {type(e).__name__}: {e}"
    | AssertionError: parse_group_ages crashed on ':' with: ValueError: invalid literal for int() with base 10: ':'
    | assert False
    | Failing test case: test_parse_group_ages_no_crash(
    |     s=':',
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         ~/LAB7/test_museum_admission_parser.py:32
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "~/LAB7/test_museum_admission_parser.py", line 31, in test_parse_group_ages_no_crash
    |     result = parse_group_ages(s)
    |   File "~/LAB7/museum_admission_parser.py", line 14, in parse_group_ages
    |     return [int(age) for age in text.split()]
    |                                 ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "~/LAB7/test_museum_admission_parser.py", line 33, in test_parse_group_ages_no_crash
    |     assert False, f"parse_group_ages crashed on {s!r} with: {type(e).__name__}: {e}"
    | AssertionError: parse_group_ages crashed on None with: AttributeError: 'NoneType' object has no attribute 'split'
    | assert False
    | Failing test case: test_parse_group_ages_no_crash(
    |     s=None,
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         ~/LAB7/test_museum_admission_parser.py:32
    +-----------------------------------
_____________________________ test_parse_visit_date_no_crash ______________________________
  + Exception Group Traceback (most recent call last):
  |   File "~/LAB7/test_museum_admission_parser.py", line 41, in test_parse_visit_date_no_crash
  |     @given(date_inputs)
  |                ^^^
  |   File "/usr/local/lib/python3.13/site-packages/hypothesis/core.py", line 2334, in wrapped_test
  |     raise the_error_hypothesis_found
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "~/LAB7/test_museum_admission_parser.py", line 44, in test_parse_visit_date_no_crash
    |     result = parse_visit_date(s)
    |   File "~/LAB7/museum_admission_parser.py", line 21, in parse_visit_date
    |     year, month, day = text.strip().split('-')
    |     ^^^^^^^^^^^^^^^^
    | ValueError: not enough values to unpack (expected 3, got 1)
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "~/LAB7/test_museum_admission_parser.py", line 46, in test_parse_visit_date_no_crash
    |     assert False, f"parse_visit_date crashed on {s!r} with: {type(e).__name__}: {e}"
    | AssertionError: parse_visit_date crashed on '' with: ValueError: not enough values to unpack (expected 3, got 1)
```

```
    | assert False
    | Failing test case: test_parse_visit_date_no_crash(
    |     s='',
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         ~/LAB7/test_museum_admission_parser.py:45
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "~/LAB7/test_museum_admission_parser.py", line 44, in test_parse_visit_date_no_crash
    |     result = parse_visit_date(s)
    |   File "~/LAB7/museum_admission_parser.py", line 21, in parse_visit_date
    |     year, month, day = text.strip().split('-')
    |                        ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'strip'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "~/LAB7/test_museum_admission_parser.py", line 46, in test_parse_visit_date_no_crash
    |     assert False, f"parse_visit_date crashed on {s!r} with: {type(e).__name__}: {e}"
    | AssertionError: parse_visit_date crashed on None with: AttributeError: 'NoneType' object has no attribute 'strip'
    | assert False
    | Failing test case: test_parse_visit_date_no_crash(
    |     s=None,
    | )
    | Explanation:
    |     These lines were always and only run by failing test cases:
    |         ~/LAB7/test_museum_admission_parser.py:45
    +-----------------------------------
========================================== short test summary info ==========================================
FAILED test_museum_admission_parser.py::test_format_visitor_name_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures. (2 su
b-exceptions)
FAILED test_museum_admission_parser.py::test_parse_group_ages_no_crash - ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-e
xceptions)
FAILED test_museum_admission_parser.py::test_parse_visit_date_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-e
xceptions)
========================================== 3 failed in 1.21s ==========================================
```

SS3 – Fixed module: `python museum_admission_parser.py`

Valid input still gives the same result; the inputs that crashed before now produce a clear validation message instead of a traceback.

```
LAB7 — zsh

$ python museum_admission_parser.py
==== Running format_visitor_name ====
Enter a visitor's full name (e.g. asha rao): asha rao
Name on ticket: Rao, Asha

==== Running parse_group_ages ====
Enter the ages in the group separated by spaces (e.g. 34 12 70): 34 12 70
Group ages: [34, 12, 70]

==== Running parse_visit_date ====
Enter the visit date as YYYY-MM-DD (e.g. 2026-11-05): 2026-11-05
Visit date (year, month, day): (2026, 11, 5)

$ python museum_admission_parser.py
==== Running format_visitor_name ====
Enter a visitor's full name (e.g. asha rao):
Invalid name: please enter at least one word.

==== Running parse_group_ages ====
Enter the ages in the group separated by spaces (e.g. 34 12 70): 34 twelve -5
Invalid ages: enter whole numbers between 0 and 120 separated by spaces.

==== Running parse_visit_date ====
Enter the visit date as YYYY-MM-DD (e.g. 2026-11-05): 2026-02-30
Invalid date: enter a real date in YYYY-MM-DD format.
```

SS4 – Hypothesis tests passing on the fixed module

```
LAB7 — zsh

$ python -m pytest test_museum_admission_parser.py -p no:cacheprovider -v
============================================ test session starts ============================================
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0 -- /usr/local/bin/python
hypothesis profile 'default'
rootdir: ~/LAB7
plugins: platformdirs-4.13.5, anyio-4.15.1, hypothesis-6.170.15
collecting ... collected 3 items

test_museum_admission_parser.py::test_format_visitor_name_no_crash PASSED                          [ 33%]
test_museum_admission_parser.py::test_parse_group_ages_no_crash PASSED                             [ 66%]
test_museum_admission_parser.py::test_parse_visit_date_no_crash PASSED                             [100%]

============================================= 3 passed in 1.37s =============================================
```

Bugs found by fuzzing and how they were fixed

Function	Minimal failing input (Hypothesis)	Root cause	Fix
format_visitor_name	None " / ' '	AttributeError: None has no .split() IndexError: parts[-1] on empty list	Return None if not a str or no words; single word returns just the surname (no dangling ', ')
parse_group_ages	None ':' (any non-digit) '-1'	AttributeError on .split() ValueError from int() Negative / unrealistic ages accepted	Type check; accept only ASCII-digit tokens (rejects signs, '_', decimals, Unicode digits); range 0–120; else return None
parse_visit_date	None ''	AttributeError on .strip() ValueError: tuple unpack when dash count ≠ 2	Type check; strict regex \d{4}-\d{2}-\d{2}; validate with datetime.date (rejects 2026-02-30, month 13); else return None

main() was also updated to print a friendly message when a function returns None, so the program never exits with a traceback.

Reflection

1. What kinds of bugs did fuzzing reveal that normal unit tests might miss?

Hypothesis immediately found inputs we never wrote tests for: **None**, the **empty string**, whitespace-only text, a lone punctuation character (":") and a negative number ("-1"). The original code assumed every input was well-formed, so these caused AttributeError, IndexError and ValueError crashes, and in the case of "-1" a silent logic bug (a negative age was accepted). Unit tests only check the cases the developer already thinks of; the fuzzer explores the whole input space and then **shrinks** each failure to the smallest reproducing example.

2. How did you fix the bugs, and what principle did you follow?

Defensive programming / input validation at the boundary: (a) check the type first, (b) check the structure (non-empty, correct number of tokens, strict YYYY-MM-DD regex), (c) check the semantics (age within 0–120, date actually exists on the calendar via datetime.date). Instead of letting an exception escape, each function returns **None** for invalid input, and main() turns that into a user-friendly error message. Validation is an allow-list (only ASCII digits are accepted) rather than trying to block every bad case.

3. What are the strengths and limitations of property-based fuzzing with Hypothesis?

Strengths: generates hundreds of varied inputs per test (Unicode, None, edge values), automatically minimises failing cases, remembers failures in its database for replay, and lets one test express a general property ("never crashes", "ages are always 0–120", "returned date is real"). **Limitations:** it only checks the properties we write – a "no crash" test alone would not have caught the negative-age logic bug, so meaningful properties matter; it is random, so rare paths may need targeted strategies (we added regex-based near-valid inputs and real dates); and it cannot tell us what the correct business rule is (e.g. the age limit) – that still needs the specification.